

Project 5: Virtual Memory Simulator

Comparison of Page Replacement Algorithms

CJ De Vault
CSc 452 – Spring 2026

May 4, 2026

Abstract

This report compares four page replacement policies—OPT, CLOCK, Rand, and NRU—using a single-level page table with 16 KiB (16 384-byte) pages and a parameterized number of physical frames. Experiments use Valgrind lackey memory traces (`gcc.trace` and `ls.trace`). OPT serves as the theoretical lower bound on page faults for each trace and frame count.

1 Introduction

Demand paging keeps only a subset of a process’s pages in RAM; on a page fault, the OS must choose which resident page to replace. The replacement policy strongly influences the page fault rate and the volume of write-back I/O for dirty pages.

2 Implementation Summary

Table 1: Simulator configuration

Item	Setting
Page size	16 KiB (16 384 bytes)
Address mapping	<code>page = address » 14</code> (32-bit addresses)
Trace events	I, L, S, M; other lines skipped
Modify (M)	One access; treated as read+write (sets dirty)
NRU refresh	<code>-r = 1000</code> references between clearing R bits
Quiet mode	<code>-q</code> suppresses per-access lines when collecting statistics

Algorithms:

- **OPT:** Evict the resident page whose next use is farthest in the future (implemented efficiently with precomputed next-use indices).
- **CLOCK:** Second-chance clock using a reference bit.
- **Rand:** Random victim among resident frames.
- **NRU:** Four classes from (R, D) ; periodic R refresh every 1000 references.

3 Methodology

1. Primary workload: `gcc.trace` — 1 711 763 memory accesses.
2. Secondary workload: `ls.trace` — 766 387 accesses.
3. Frame counts: 8, 16, 32, 64.
4. NRU: fixed refresh interval `-r 1000` for all runs (reasonable middle ground between clearing R too often vs. letting R bits go stale).

Example commands

```
./vmsim -q -n 8 -a opt gcc.trace
./vmsim -q -n 16 -a clock gcc.trace
./vmsim -q -n 8 -a nru -r 1000 gcc.trace
# repeat sweeps over frame counts and algorithms; same pattern for ls.trace
```

4 Results: `gcc.trace`

4.1 Page faults

Table 2: Total page faults (`gcc.trace`, NRU with `-r 1000`)

Frames	OPT	CLOCK	Rand	NRU
8	11 357	20 455	27 469	44 116
16	2581	6068	7643	7544
32	428	844	1265	1254
64	113	144	237	508

Fault counts drop sharply as frames increase; OPT is lowest at each column.

4.2 Fault ratio vs. OPT

Table 3: Ratio of faults to OPT (`gcc.trace`)

Frames	CLOCK/OPT	Rand/OPT	NRU/OPT
8	1.80×	2.42×	3.89×
16	2.35×	2.96×	2.92×
32	1.97×	2.96×	2.93×
64	1.27×	2.10×	4.50×

NRU at 64 frames is comparatively worse than CLOCK/Rand on this trace; worth discussing interaction between refresh, class ordering, and locality.

Table 4: Total writes to disk (`gcc.trace`)

Frames	OPT	CLOCK	Rand	NRU
8	1442	2954	5327	1055
16	309	856	1603	367
32	32	88	226	2
64	3	5	29	0

4.3 Writes to disk (dirty evictions)

5 Results: `ls.trace`

Table 5: Total page faults (`ls.trace`)

Frames	OPT	CLOCK	Rand	NRU
8	7152	13 484	19 102	27 978
16	1544	4317	4719	4004
32	258	419	651	662
64	104	158	182	217

Table 6: Fault ratio vs. OPT (`ls.trace`)

Frames	CLOCK/OPT	Rand/OPT	NRU/OPT
8	1.89×	2.67×	3.91×
16	2.80×	3.06×	2.59×
32	1.62×	2.52×	2.57×
64	1.52×	1.75×	2.09×

On `ls.trace`, NRU at 64 frames is not an outlier the way it is on `gcc.trace`; workloads differ.

6 Figures: Page Faults vs. Frames

Page Faults vs. Frames (gcc.trace)

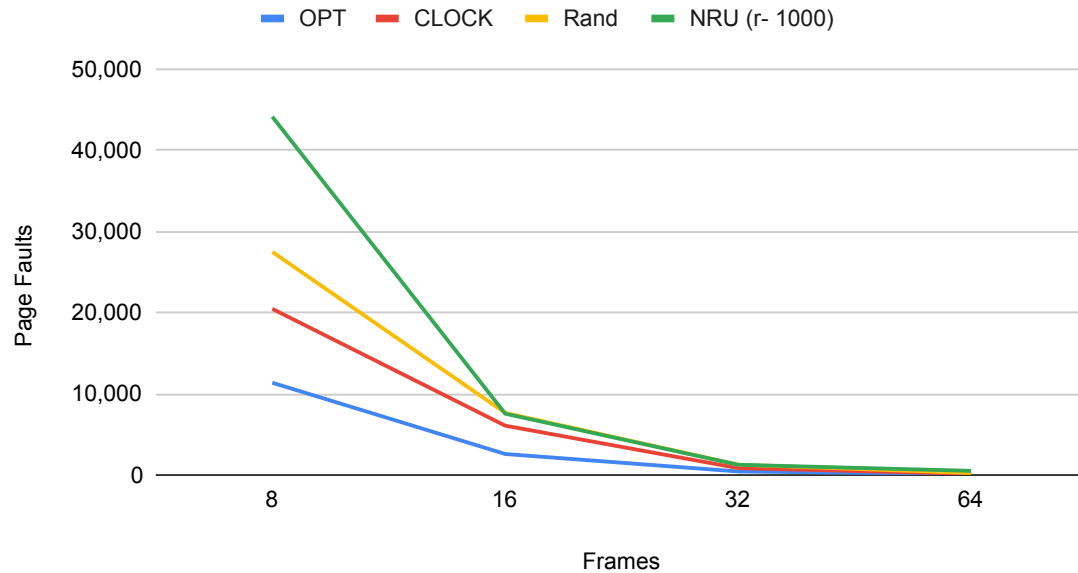


Figure 1: Page faults vs. number of frames (`gcc.trace`). NRU uses `-r = 1000`.

Page Faults vs Frames (ls.trace)

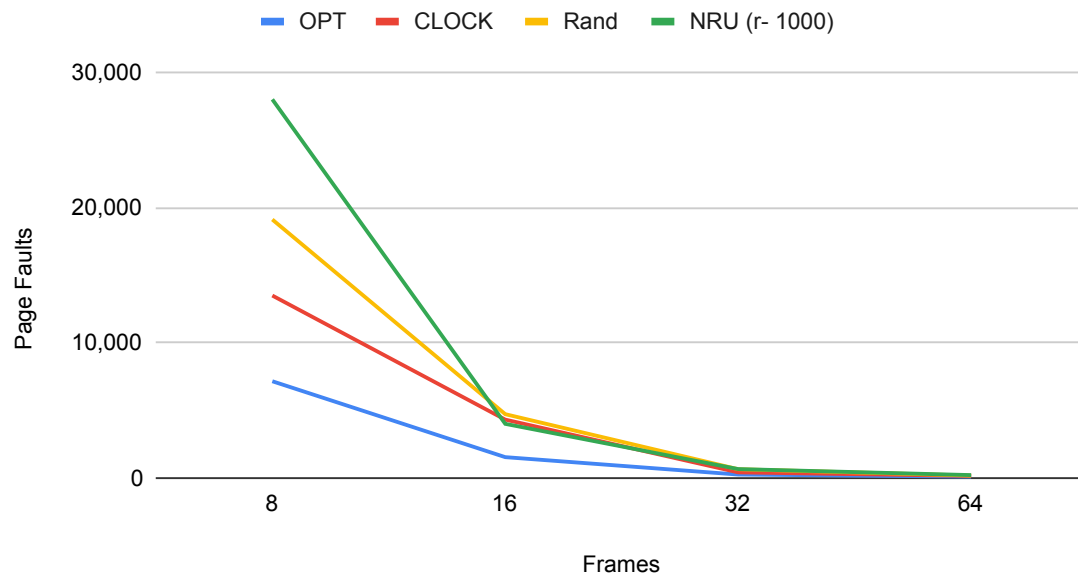


Figure 2: Page faults vs. number of frames (`ls.trace`). NRU uses `-r = 1000`.

7 Analysis and Conclusions

7.1 Interpreting the comparison to OPT

OPT is not a competing “policy” for a real kernel—it requires knowledge of the entire future reference stream—but it *is* the right baseline: for a fixed trace and a fixed number of frames, no online algorithm can have fewer page faults. The practical question is how close a realizable policy comes on realistic workloads.

On `gcc.trace`, at 8 frames, CLOCK (20 455 faults) and Rand (27 469) are about $1.8\times$ and $2.4\times$ OPT respectively, while NRU (44 116) is nearly $3.9\times$ OPT. As memory increases, the story changes. At 32 frames, CLOCK, Rand, and NRU are all in a similar band—roughly 2.0 – $2.9\times$ OPT—suggesting that a larger cache shrinks the *structural* gap for these heuristics on this trace, at least until the working set pressure changes. The striking outlier is NRU at 64 frames: 508 faults vs. only 113 for OPT and 144 for CLOCK. The ratio NRU/OPT jumps to $4.5\times$ even though Rand only sits at $2.1\times$. That is not a small “implementation detail”; it shows that a policy with a fixed refresh and class-based eviction can be *very* sensitive to the interaction between the refresh period, the R/D classification, and the particular reference pattern of a long-running compile.

On `ls.trace`, the same NRU configuration does *not* show the same pathology: at 64 frames, NRU’s ratio to OPT is about $2.1\times$ —in line with Rand. The same algorithm, the same `-r = 1000`, but a different program and locality. This is the main lesson is no single `-r` or static classification scheme is guaranteed to track “recency” well on every trace. A report that only used `gcc.trace` might recommend against NRU; a report that only used `ls.trace` would understate that risk.

7.2 CLOCK vs. Rand

CLOCK is a standard approximation to LRU: it uses a single reference bit and a circular scan, so the cost per replacement is bounded and the implementation is easy to reason about. Here, CLOCK consistently beats Rand on both traces. At 8 frames on `gcc.trace`, for example, CLOCK’s fault count is about 25% lower than Rand’s. Random replacement has a certain appeal (no extra bits, no pointer manipulation), but it ignores both recency and frequency, and the trace shows the cost. If the random number generator is not fixed to a seed, a single run of Rand is one sample from a distribution; the gap to CLOCK could vary, but it is unlikely to close entirely without “luck.”

7.3 NRU, refresh, and the gcc outlier at 64 frames

NRU evicts from the lowest (R, D) class; the periodic refresh clears R so that pages that are not re-touched within the refresh window appear “not recently used.” When that classification aligns with the real reuse horizon of the program, NRU performs well; when it misaligns—for example, if important pages lose their R bit just before they are needed again—faults can spike. The poor NRU result at 64 frames on `gcc.trace` while NRU does well at 32 on the same trace suggests that more frames are not always easier for NRU: with more resident pages, refresh-driven misclassification or victim choice within a class can interact badly with the phase behavior of the workload. Tuning `-r` (or using an adaptive refresh) would be the next step in a longer study; the assignment’s fixed refresh is enough to show that NRU is *not* “set and forget.”

7.4 Writes to disk

Fault counts are the headline metric, but dirty evictions matter for I/O and latency. On `gcc.trace`, OPT’s dirty writebacks fall from 1442 at 8 frames to 3 at 64, reflecting fewer evictions overall. NRU

shows very few dirty evictions at 32/64 in these runs because the policy often evicts pages that are not dirty at eviction time; that can be good for I/O even when fault counts are high. Interpreting dirty counts alongside faults gives a fuller picture than faults alone.

7.5 Scaling with the number of frames

Doubling frames from 8 to 16 cuts OPT faults on `gcc.trace` by more than a factor of four (11 357 → 2581); doubling again to 32 yields another large drop (428). By 64, OPT is already near 113 faults—the curve is strongly convex. That pattern is typical: the first additional frames buy the most because they capture the hottest portion of the working set; later frames capture long-tail cold pages that matter less per dollar of RAM. Policies that track locality well (here, CLOCK relative to Rand) capture more of that benefit at moderate frame counts.

7.6 What would use in a real OS?

OPT is a mathematical benchmark, not a shipping policy. Between the practical policies in this assignment:

- **CLOCK** (and related approximations to LRU) emerge as the strongest default story from these traces: predictable implementation, bounded work per eviction, and fault counts consistently closer to OPT than Rand.
- **Rand** is simple but wasteful on structured workloads like compilation and file utilities; it may still appear in specialized contexts where deterministic behavior is undesirable, but it is a poor general-purpose choice when reference locality exists.
- **NRU** can be competitive (see `ls.trace` and moderate frame counts on `gcc.trace`) but requires careful tuning of refresh and is trace-sensitive; it illustrates why production systems often invest in *global* replacement heuristics and adaptive behavior rather than a single static scheme.

If forced to pick one policy from this experiment for a teaching kernel or a small embedded OS without heavy machinery, **CLOCK** is the most defensible balance of performance, complexity, and stability across both traces.

7.7 Limitations

These conclusions rest on two traces, one NRU refresh value, and (for Rand) a single stochastic run unless a fixed seed is used. Additional traces and sweeps over `-r` would be needed to generalize claims beyond “on these workloads, with these parameters.”

Acknowledgments / Data

Memory traces were produced with Valgrind lackey as provided for the course. The `-q` flag was used to avoid printing ~1.7M per-access lines for `gcc.trace` when collecting summaries.